BIRZEIT UNIVERSITY

Computer Organization and Microprocessor (ENCS2380)

Project: Single Cycle Processor Design

Student's names:

Tasneem Shelleh ID: 1220439.

Lana Sayes ID: 1220785.

Instructor's name: Ismail Khater

Section : 1

Date:12-6-2024

Abstract:

A computer comprises three fundamental components: the CPU (Central Processing Unit) which serves as the brain of the system, executing instructions and performing calculations, main memory :such as RAM (Random Access Memory), which provides temporary storage for data and instructions, and input/output: which enable communication between the computer and the user as well as other external devices. Each plays a crucial role, with the CPU housing the ALU (Arithmetic Logic Unit), control unit, and buses for interconnection.

In this project , we’re asked to create Single Cycle Processor Design using Logisim.

We designed a 32-bit processor with 16-bit instructions, there are four instruction types, R-type, I-type, S-type, and J-type.

Contents

Table of Tables

Table of Figure

Register Circuit:

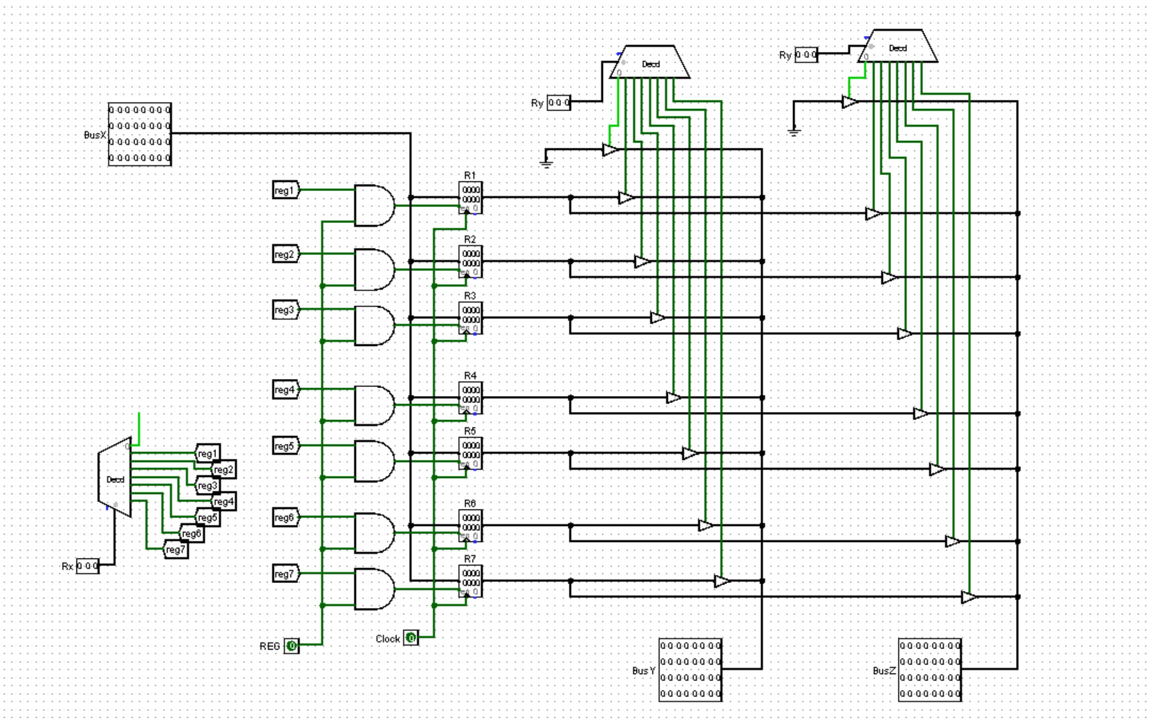
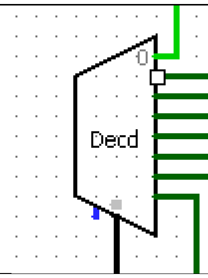
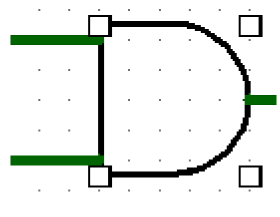
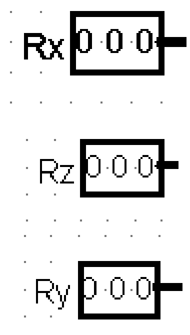
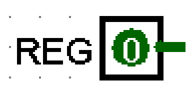
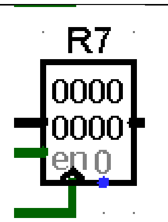
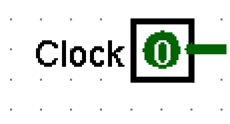


Figure 1: Reg circuit

Registers are tiny, super-fast storage locations, such that we use it in the CPU to hold and store data, addresses, and control information. Such that, we use R1 – R7 in the circuit that we choose which reg to use according to X, Y and Z selectors that they control the decoders to operate one reg.

Logisim tool	**Component**	**Drawing**	**Description**
Plexers	DECODER		To specify which reg to choose according to selection
Gates	AND		ANDing reg from R1 – R7 with REG
Wiring	TUNNEL		to connect between components
Pin	Input		Working as selection
Pin	Input		Work as enable
Memory	Register		To store data
pin	input		Work as clock

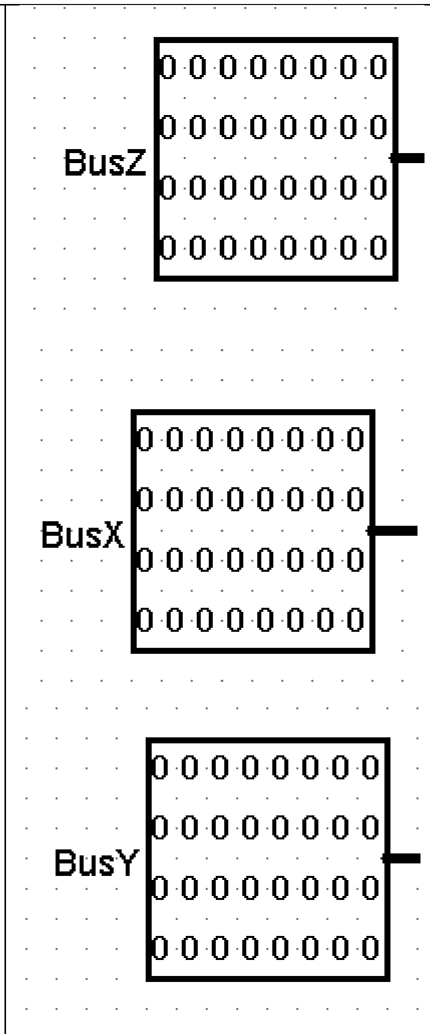
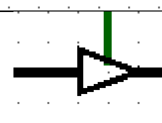
Pin	Input		
Gates	Buffer		Delay

Table 1: Reg component

ALU Circuit

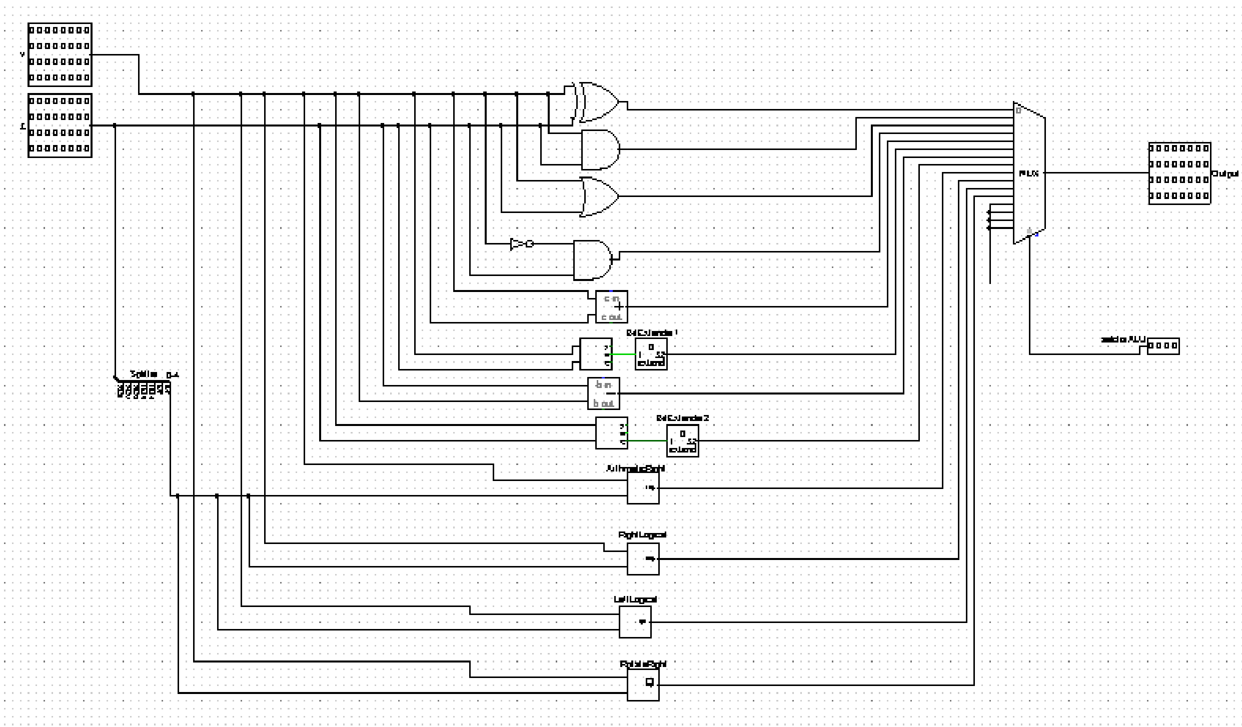
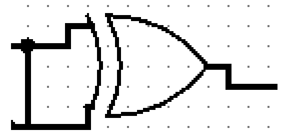
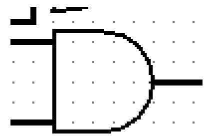
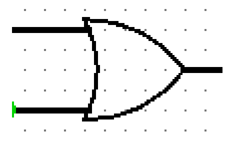
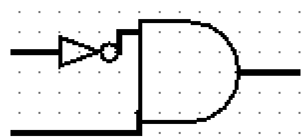
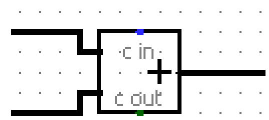
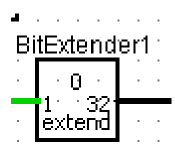
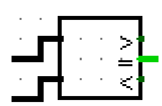
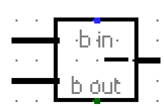


Figure 2: ALU circuit

The ALU is an important part of the CPU that performs arithmetic and logical operations. We use to implement it a 16 bit MUX and basic gates and arithmetic gates, that every gate express one of the 12 operations (the last 4 input in multiplexer are unused).Inputs and output are 32 bits.

Logisim tool	Component	Drawings	Description
Gates	XOR		To implement XOR
Gates	AND		To implement AND
Gates	OR		To implement OR
Gates	NOT,AND		To implement CAND
Arithmetic	ADDER		To implement ADD
Wiring	Bit extender		To extend bits
Arithmetic	Comparator		To implement SEQ, SLT
Arithmetic	SUBTRACTOR		To implement NAND

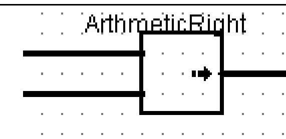
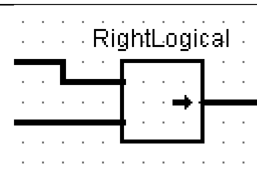
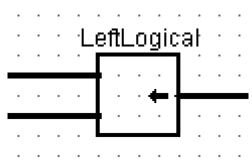
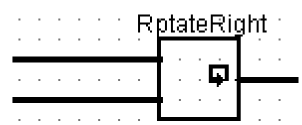
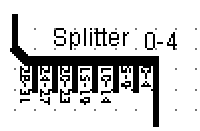
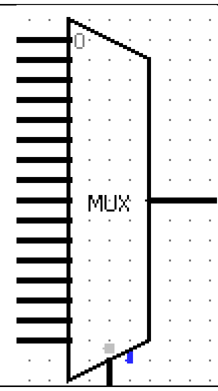
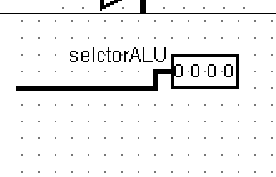
Arithmetic	Shifter		To implement SRA
Arithmetic	Shifter		To implement SRL
Arithmetic	Shifter		To implement SLL
Arithmetic	Shifter		To implement ROR
Wiring	Splitter		Split bits
Arithmetic	Multiplexer		Choosing the wanted operation depending0 on selector
Pin	Input		Bit selector

Table 2: ALU component

Control Unit:

Main & ALU Control Unit :

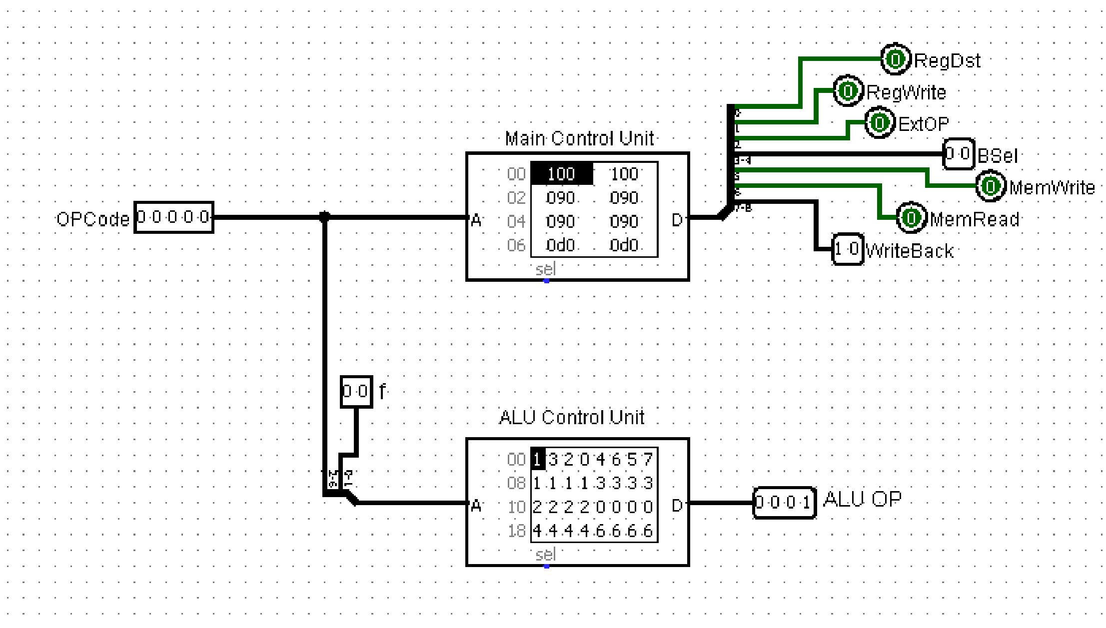


Figure 3: Main & ALU Control Unit

The Control Unit is a key part of the CPU that fetches, decodes, and executes instructions. Such that, it chooses which instruction to execute according to OP code for main control unit, OP and f code for ALU control unit, that it chooses from the ROM which microcode according to them, then the wanted selector will operate.

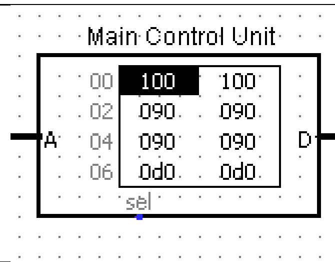
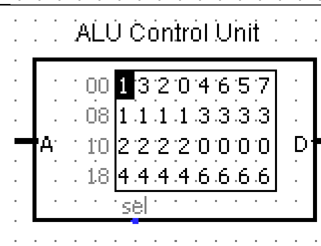
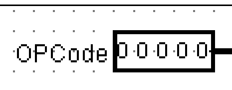
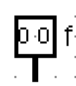
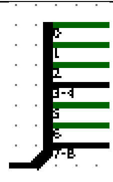
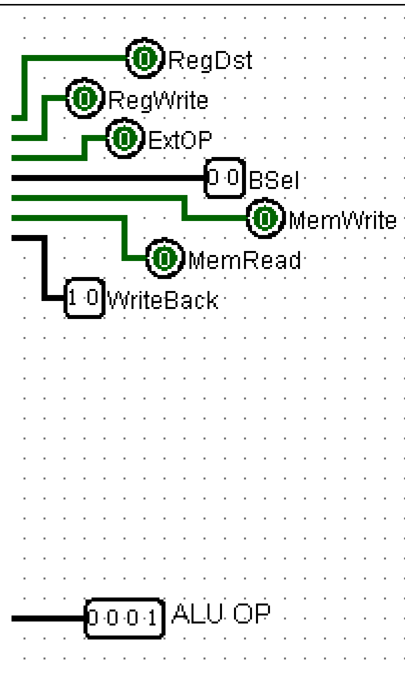
Logisim tool	Component	Drawing	Description
Memory	ROM		To store the microcode of the instructions in the main data path
Memory	ROM		To store the microcode of the instructions in ALU
Pin	Input		To enter op code
Pin	Input		To enter f code
Wiring	Splitter		To split the bits
Pin	Output		To execute the operations according to them such that they depend on op and f code

Table 3: main & ALU component

Control Signals:

-RegDst(1bit) --> which Reg is the Dst

0> Rx

1> R7

-RegWrite(1bit) --> Src Written on Reg

0> No Write On Reg

1> Write On Reg

-ExtOP(1bit) --> Extension OP Code (Zero or Sign)

0> Zero extend Imm5

1> Sign extend Imm5

-BSel(2bits) --> Bus Selection (which Bus we do the op. with it)

00> BusZ

01> Imm5

10> Imm2,Imm3

11> Imm11

-MemWrite(1bit) --> Write on Memo. at specific address

0> No Write

1> Write

-MemRead(1bit) --> Read from Memo. at specific address

0> No Read

1> Read

-WriteBack(2bits) --> the result will be taken from :

00> ALU Result

01> Data Memory

10> PC+2

11> case not consider

Main Control Signals :

Inst.	RegDst	RegWrite	ExtOP	BSel	MemWrite	MemRead	WriteBack	OP	Hex
AND	0	1	0	00	0	0	00	0	0x080
CAND	0	1	0	00	0	0	00	0	0x080
OR	0	1	0	00	0	0	00	0	0x080
XOR	0	1	0	00	0	0	00	0	0x080
ADD	0	1	0	00	0	0	00	1	0x080
NADD	0	1	0	00	0	0	00	1	0x080
SEQ	0	1	0	00	0	0	00	1	0x080
SLT	0	1	0	00	0	0	00	1	0x080
ANDI	0	1	0	01	0	0	00	2	0x090
CANDI	0	1	0	01	0	0	00	3	0x090
ORI	0	1	0	01	0	0	00	4	0x090
XORI	0	1	0	01	0	0	00	5	0x090
ADDI	0	1	1	01	0	0	00	6	0x0d0
NADDI	0	1	1	01	0	0	00	7	0x0d0

SEQI	0	1	1	01	0	0	00	8	0x0d0
SLTI	0	1	1	01	0	0	00	9	0x0d0
SLL	0	1	0	01	0	0	00	10	0x090
SRL	0	1	0	01	0	0	00	11	0x090
SRA	0	1	0	01	0	0	00	12	0x090
ROR	0	1	0	01	0	0	00	13	0x090
BEQ	0	1	1	01	0	0	00	14	0x0d0
BNE	0	1	1	01	0	0	00	15	0x0d0
BLT	0	1	1	01	0	0	00	16	0x0d0
BGE	0	1	1	01	0	0	00	17	0x0d0
LW	0	1	1	01	0	1	00	18	0x0d4
SW	0	0	0	11	1	0	01	19	0x039
J	0	1	1	10	0	0	10	20	0x0e2
JAL	1	1	1	10	0	0	00	21	0x1e0

Table 4: main control unit signals

ALU Control Signals:

Inst.	OP(5bits)	f	ALUOP	Hex
XOR	00000	11	0000	0x03
AND	00000	00	0001	0x00
OR	00000	10	0010	0x02
ADD	00001	00	0011	0x04
CAND	00000	01	0100	0x01
NADD	00001	01	0101	0x05
SEQ	00001	10	0110	0x06
SLT	00001	11	0111	0x07
SRA	01100	xx=00	1000	0x30
	01100	xx=01	1000	0x31
	01100	xx=10	1000	0x32
	01100	xx=11	1000	0x33
SRL	01011	xx=00	1001	0x2C
	01011	xx=01	1001	0x2d
	01011	xx=10	1001	0x2e
	01011	xx=11	1001	0x2f
SLL	01101	xx=00	1011	0x34
	01101	xx=01	1011	0x35
	01101	xx=10	1011	0x36
	01101	xx=11	1011	0x37

Table 5: ALU control unit signals

PC Control Unit :

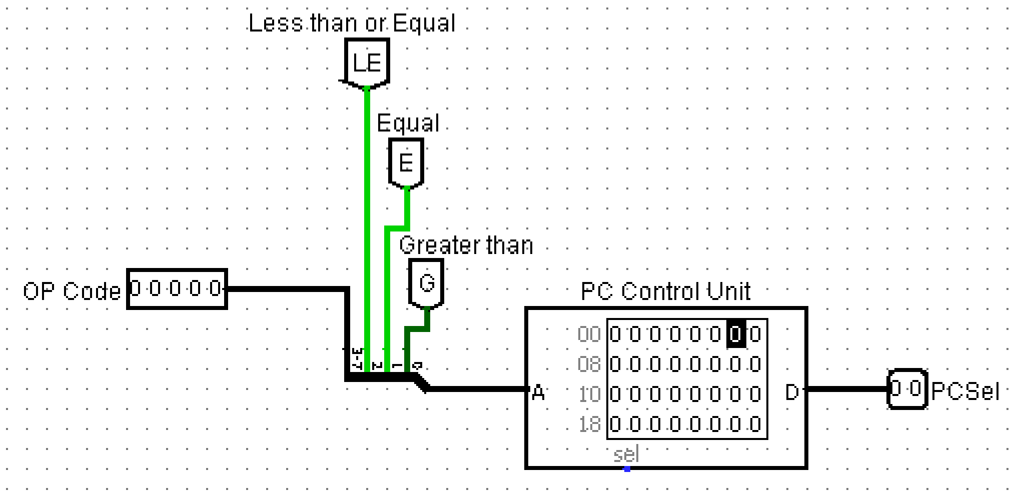


Figure 4: PC Control Unit

The Program Counter (PC) is a register in CPU that responsible for holding the address of the next instruction to be executed. The PC control unit is to ensures the correct sequence of operations, and manages jumps and branches.

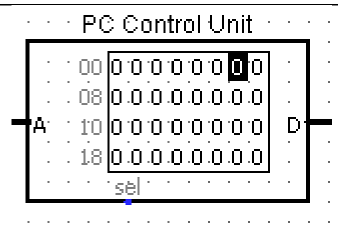
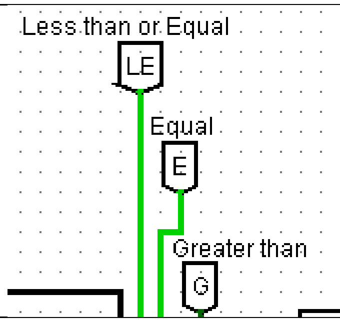
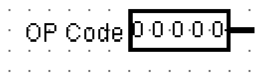
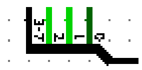
Logisim tool	Component	Drawing	Description
Memory	ROM		To store the microcode of the operations in PC
Wiring	Tunnel		To connect between component
Pin	Input		To enter op code
Wiring	Splitter		To split the bits
Pin	Output		To execute the operations according to it such that they depend on op

Table 6: PC control unit component

PC Control Signals :

Inst.	OP (5bits)	LE	E	G	PCSel	Hex
BEQ	01110	1	1	0	01	0x76
BNQ	01111	X = 0	0	X=1	01	0x79
	01111	X=1	0	X=0	01	0x7C
BLT	10000	0	0	1	01	0x81
BGE	10001	1	X=0	0	01	0x8C
	10001	1	X=1	0	01	0x8E
J	10100	X=0	X=0	X=1	10	0xA1
	10100	X=1	X=1	X=0	10	0xA6
	10100	X=1	X=0	X=0	10	0xA4
JAL	10101	X=0	X=0	X=1	10	0xA9
	10101	X=1	X=1	X=0	10	0xAE
	10101	X=1	X=0	X=0	10	0xAC

Table 7: PC control unit signals

Branch:

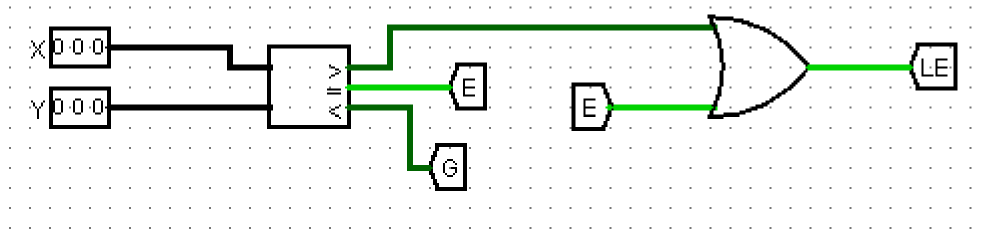


Figure 5: Branch

A branch is an instruction that changes the flow of execution in a program by directing the CPU to jump to a different instruction address. Such that in this CPU the branch compare between reg X and reg Y values that

if (X == Y) ? (PC += sign_extend(Imm5<<1))

else if (X != Y)? (PC += sign_extend(Imm5<<1))

else if (X < Y)? (PC += sign_extend(Imm5<<1))

else if (X >= Y)? (PC += sign_extend(Imm5<<1))

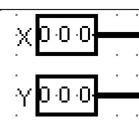
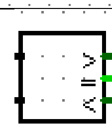
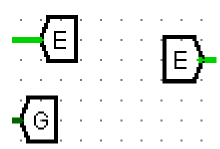
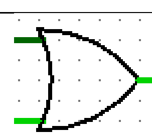
Logisim tool	**Component**	**Drawing**	**Description**
Pin	Input		To enter X ,Y values
Arithmetic	Comparator		To compare between X ,Y
Wiring	Tunnel		To connect between component
Arithmetic	OR		To implement Equal OR Less than

Table 8: Branch component

Data Path:

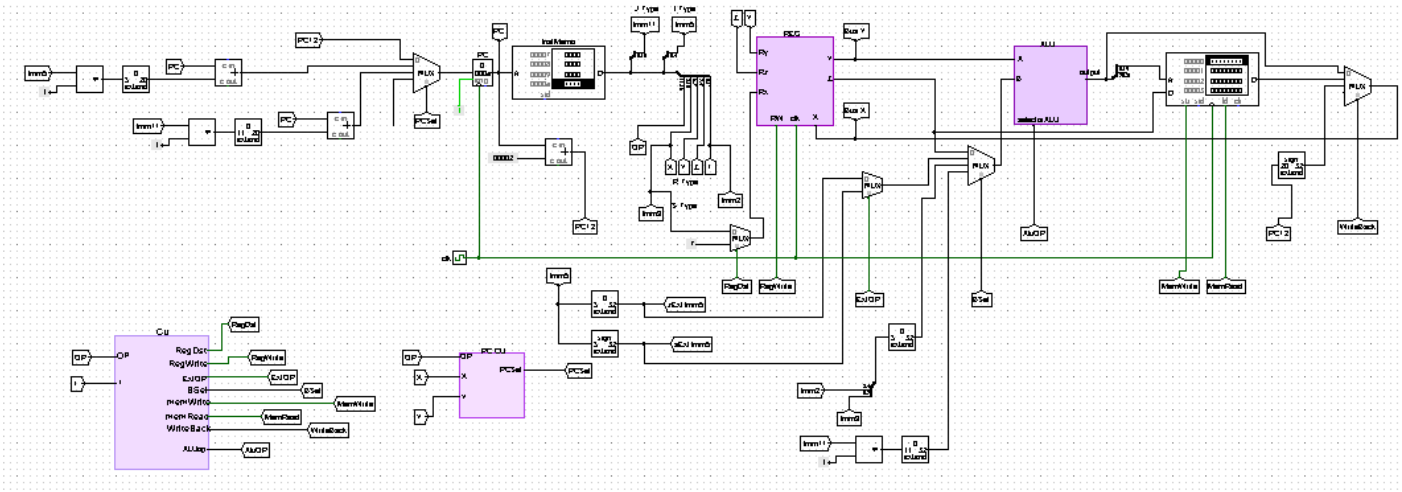
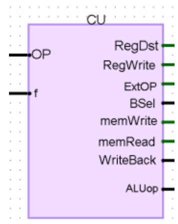
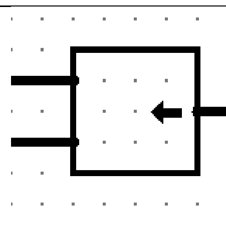
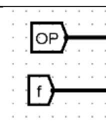
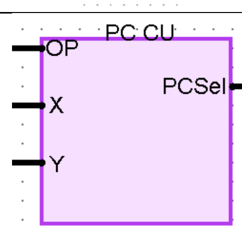
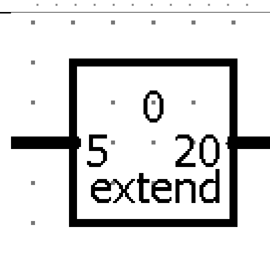
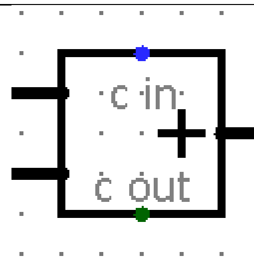
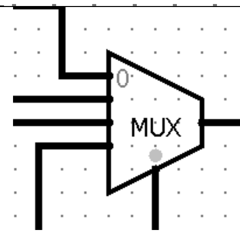
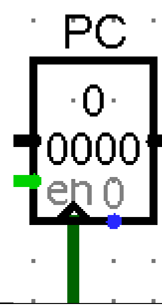
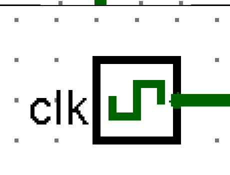
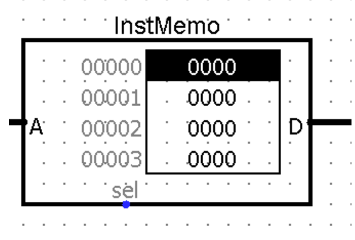
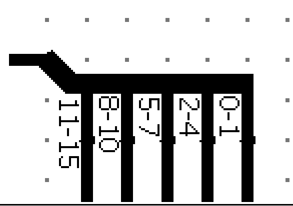


Figure 6: Data path

The data path in a CPU is the network of hardware components and connections that manage the data during instruction execution. Such that we implement it using registers, the ALU, multiplexers, buses, memory, and the control unit. We can say that it is the main component of the CPU, that it control the instructions in CPU.

Logisim tool	Component	Drawing	Description
	Control Unit		The control unit controls the data path within the processor, directing the data between registers, the ALU, memory, and other components.
Arithmetic	Shifter		Shifting Bits
Wiring	Tunnel		To connect between component
	PC CU		manage the execution of instructions
Wiring	Bit Extender		Extending bits

Arithmetic	ADDER		Adding pc and extended tunnel
Arithmetic	Multipexer		Choosing a specific operation from many depending on selector
Memory	Register		Temporary data storage
Wiring	Clock		providing timing control and synchronization
Memory	ROM		To store the microcode of the operations in PC
Wiring	Splitter		Split bits

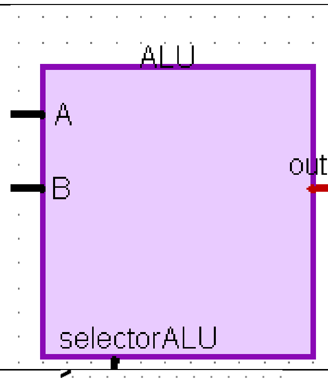
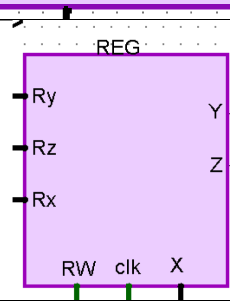
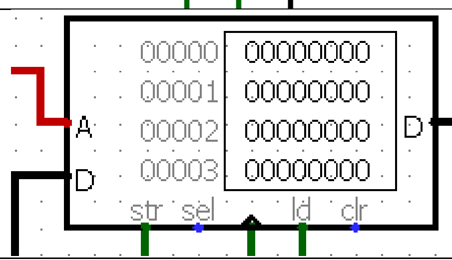
	ALU		handle all the required operations.
	REG		display the contents of ALL registers from R1 to R7
Memory	RAM		Temporary storage (ALU ,Reg)

Table 9: data path component

PC :

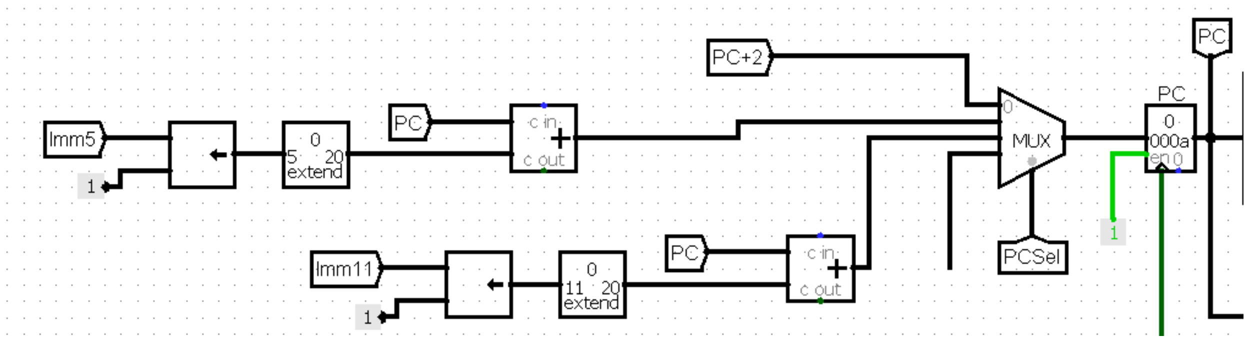


Figure 7: PC

PC work as a register that responsible for holding the address of the next instruction to be executed. So that it is increment with each clock or cycle to the next address.

There is 3 ways to increment the PC:

- PC = PC + 2 : which is the default way
- PC = PC + sign_extend(Imm5<<1) : which is for taken branches (BEQ, BEN, BLT, BGE)
- PC = PC + sign_extend(Imm11<<1) : which is for jump (J, JAL)

To implement this, we use multiplexer (2bit selection), adder, shifter, bit extender and tunnels.

ALU in data path:

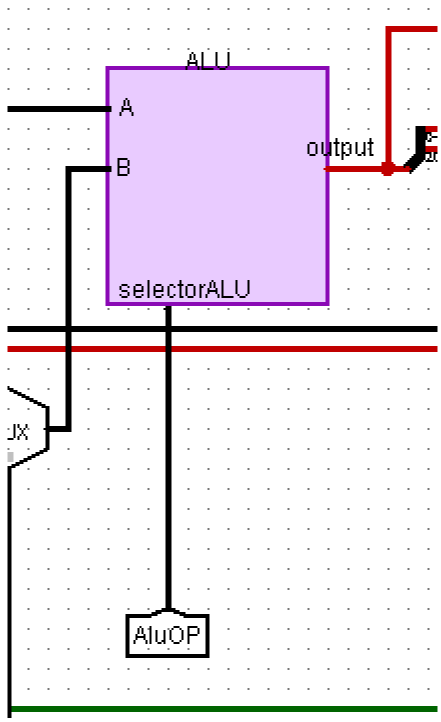


Figure 8: ALU

The ALU is an important part of the CPU that performs arithmetic and logical operations. It is implemented when the ALUOP which is the selector in the ALU circuit enabled. It should be do the arithmetic and logical operations on the values from registers according to the instructions that is stored at the ROM.

- **Team work :**

Phase one:

Reg circuit : Tasneem

ALU circuit : Lana and Tasneem

Phase two:

Circuit installation : Lana and Tasneem

PC and Branch : Tasneem

Phase three:

Main control unit : Tasneem and Lana

ALU control unit : Tasneem

PC control unit : Tasneem

Report: Tasneem and Lana